

GMD2

Lecture 06: Super Mario Bros

Today's Goal



Today's Topics

- Procedural Level Generation
- State Pattern
- Camera
- Platformer Physics
- Basic AI
- Powerups

Source Code

- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-platformer>

Level Generation

```
<Tilemap>  
  <Tileset region="0 40 80 80" tileWidth="20" tileHeight="20">images/atlas</Tileset>  
  <Tiles>  
    00 01 02 01 03  
    04 05 06 05 07  
    08 09 10 09 11  
    04 09 09 09 07  
    12 13 14 13 15  
  </Tiles>  
</Tilemap>
```



00	01	02	01	03
04	05	06	05	07
08	09	10	09	11
04	09	09	09	07
12	13	14	13	15

Procedural Level Generation

```
public GameLevel Generate(int columns, int rows)
{
    tilemap = new(Tileset, columns, rows);

    int groundHeight = 2;
    for (int x = 0; x < columns; x++)
    {
        for (int y = 0; y < Tilemap.Rows; y++)
        {
            if (y >= Tilemap.Rows - groundHeight)
            {
                Tilemap.SetTile(x, y, 1);
            }
        }
    }
    return tilemap;
}
```



00	00	00	00	00
00	00	00	00	00
00	00	00	00	00
01	01	01	01	01
01	01	01	01	01

Tilemap Update

- Tilemap up until now have been an array of integer ids
 - Each pointing to an index in the Tileset (i.e. TextureRegion)
- Now it is an array of Tiles
 - Why?
 - Our tilemap is getting smarter!
 - We need to store more information than just the tile graphic!

Tile

```
public readonly struct Tile(int graphicId = -1, int topperId = -1, bool isSolid = false)
{
    public static readonly Tile Empty = new();

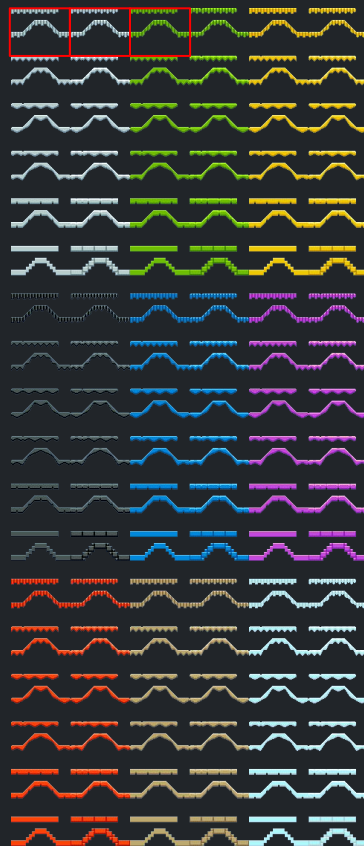
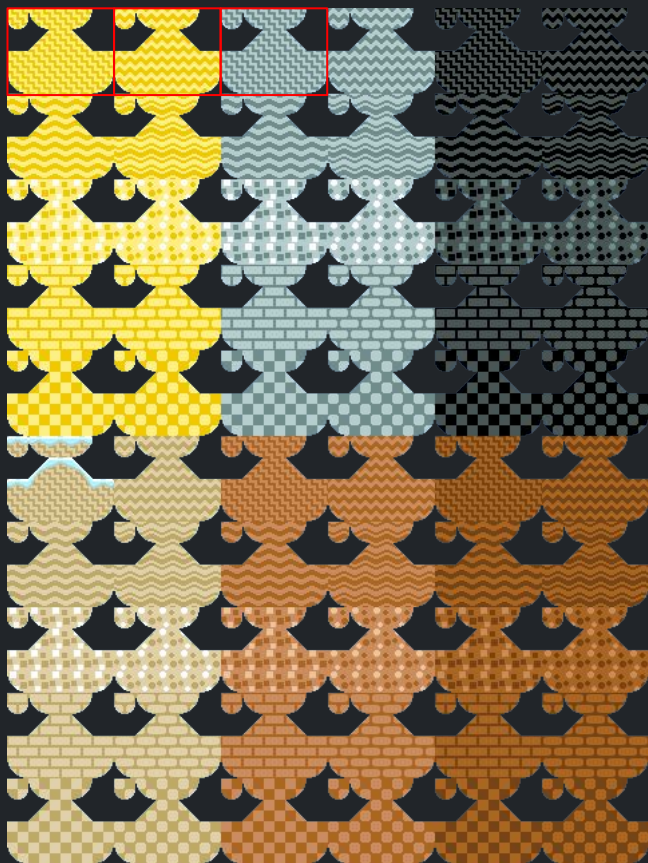
    public int GraphicId { get; init; } = graphicId;
    public int TopperId { get; init; } = topperId;
    public bool IsSolid { get; init; } = isSolid;

    public bool HasTopper => TopperId >= 0;

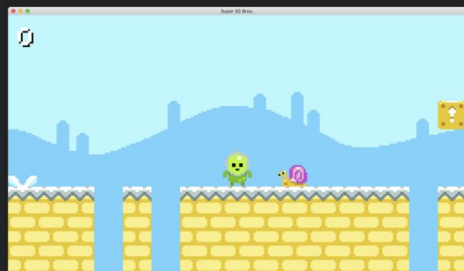
    public Tile() : this(-1, -1, false) { }

    public Tile(int graphicId, bool isSolid) : this(graphicId, -1, isSolid) { }
}
```

Tilesets



GameLevel



LevelMaker

LevelMakerBase

- SimpleLevelMaker
- FlatLevelMaker
- PillarLevelMaker
- PitlevelMaker
- ComplexLevelMaker



(This is essentially the strategy design pattern)

LevelMaker Implementation

DEMO!

Character State - Enum Solution

```
void Heroine::handleInput(Input input)
{
    switch (state_)
    {
        case STATE_STANDING:
            if (input == PRESS_B)
            {
                state_ = STATE_JUMPING;
                yVelocity_ = JUMP_VELOCITY;
                setGraphics(IMAGE_JUMP);
            }
            else if (input == PRESS_DOWN)
            {
                state_ = STATE_DUCKING;
                setGraphics(IMAGE_DUCK);
            }
            break;

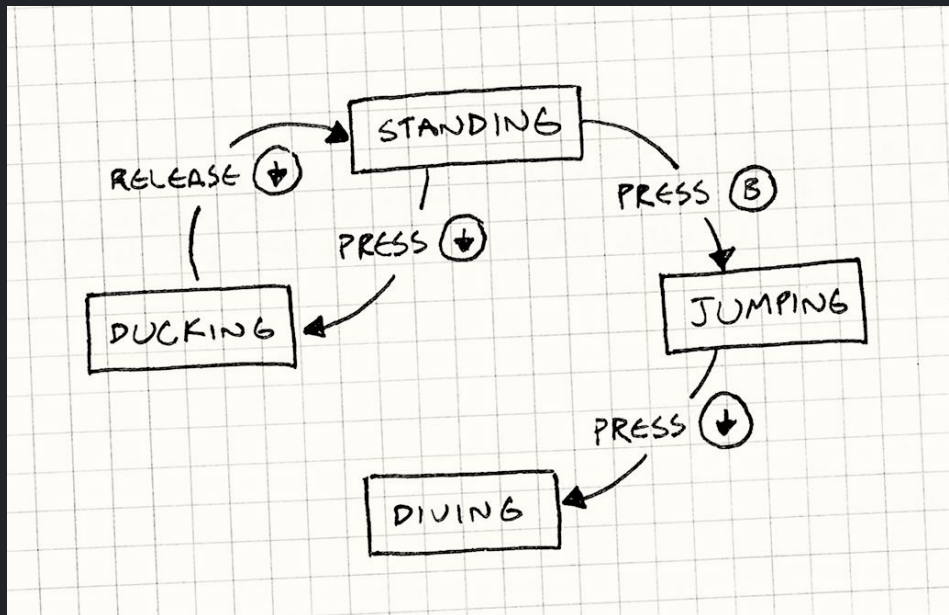
        case STATE_JUMPING:
            if (input == PRESS_DOWN)
            {
                state_ = STATE_DIVING;
                setGraphics(IMAGE_DIVE);
            }
            break;

        case STATE_DUCKING:
            if (input == RELEASE_DOWN)
            {
                state_ = STATE_STANDING;
                setGraphics(IMAGE_STAND);
            }
            break;
    }
}
```

- Why is this better?
 - You can't encode invalid state!
- Why is it still (potentially) problematic?
 - What if states have state?
 - E.g. a charge attack while ducking!
 - Doesn't scale well...

State Pattern

- Allows an object to change its behaviour based on an internal state
- State-specific behavior is defined independently
 - Adding new states does not impact the behavior of existing states



Player State Implementation

DEMO!

Game State Implementation

DEMO!

Snail States (Aka Basic AI)

- Base
 - Handles collision detection and resolution, gravity, ground detection
- Idle
 - Starting state, plays idle animation (duration)
- Move
 - Moves around, reverses at edge or when $\text{velocity.x} = 0$
- Chase
 - Moves towards player on x.axis



Snail State Implementation

DEMO!

Camera

- Why do we need a camera now and not for the previous games?
- Performs two matrix translations based on target pos
 - Shift world + Center
- In this game we only need tracking on the x-axis
- Gets clamped playerX
 - Based on tilemap dimensions

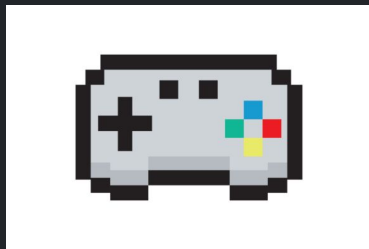


Camera Implementation

DEMO!

Input

- We now have a GameController class that abstracts away hardcoded keys for “actions”
- This is NOT the Command Pattern!
 - why not?



Player Hitbox

- Player is same width as tiles, this makes it hard to fall down pillars
 - The player can run across a pit without falling down!
- We shrink the hitbox/collider with 2px on each side
 - Now you gotta jump those pits!



Tile Collision (up/down)



Tile Collision (left/right)



Collision Response

```
private void SnapToRight(float x)
{
    Player.Position = new Vector2(x - Player.Sprite.Width + Player.HitboxInset, Player.Position.Y);
    Player.Velocity = new Vector2(0, Player.Velocity.Y);
}

private void SnapToLeft(float x)
{
    Player.Position = new Vector2(x - Player.HitboxInset, Player.Position.Y);
    Player.Velocity = new Vector2(0, Player.Velocity.Y);
}

private void SnapToBottom(float y)
{
    Player.Position = new Vector2(Player.Position.X, y - Player.Sprite.Height);
    Player.Velocity = new Vector2(Player.Velocity.X, 0);
}

private void SnapToTop(float y)
{
    Player.Position = new Vector2(Player.Position.X, y);
    Player.Velocity = new Vector2(Player.Velocity.X, 0);
}
```

IsOnGround

```
protected bool IsOnGround()
{
    Rectangle hitbox = Player.Bounds;

    // Check Tilemap
    bool onTile = Player.Tilemap.IsSolidAt(hitbox.Left + CollisionInset, hitbox.Bottom + 1) ||
        Player.Tilemap.IsSolidAt(hitbox.Right - CollisionInset, hitbox.Bottom + 1);

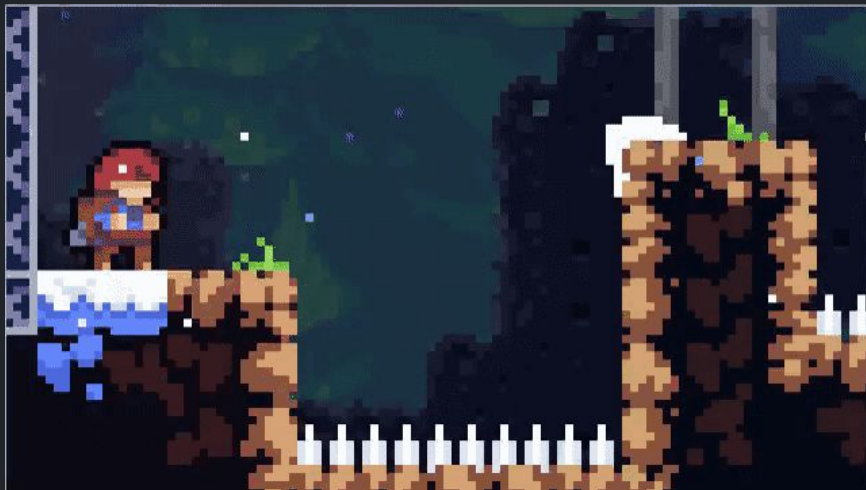
    if (onTile) return true;

    // Check solid Entities
    Rectangle sensor = ExpandDirectional(hitbox, 0, 1);
    return Player.Level.Entities.Any(e => e.Active && e.IsSolid && e.Collidable && sensor.Intersects(e.Bounds));
}
```

Tilemap Additions

```
public Tile PointToTile(Vector2 point)
public Vector2 TileToPoint(int column, int row)
public bool IsSolidAt(float x, float y)
public float GetTileLeft(float x)
public float GetTileRight(float x)
public float GetTileTop(float y)
public float GetTileBottom(float y)
private Tile? GetTileAt(float x, float y)
```

Coyote Time



Platformer Physics Performance

- Do we need collision checks between the player and every single tile?
 - No! The grid is static!
 - It is a simple “lookup” - it takes the same time in a level of 10 or 10.000 tiles
 - $O(n) \rightarrow O(1)$

```
public bool IsSolidAt(float x, float y)
{
    return GetTileAt(x, y)?.IsSolid ?? false;
}
```

- In our game we are only checking collisions between the player and other entities
 - $O(n)$
- If we naively checked collisions between all entities in our game we risk $O(n^2)$
 - $n * (n - 1) / 2$

Platformer Physics Implementation

DEMO!

Entitites

- Represent any “thing” in our game
 - Player, Snail, Boxes, Powerups, etc.
- Separate from the tiles in our map
 - Entities may not align perfectly with the tilemap
- Can contain states just like the game
 - States can affect input handling (e.g. player) or decision-making (e.g. snail)
- Some engines may adopt an Entity-Component System
 - Everything is an Entity
 - Entities are containers of components
 - Components ultimately drive behavior
 - Component Design Pattern!



Entity

```
public interface IEntity
{
    public Vector2 Position { get; set; }
    public Rectangle Bounds { get; }
    public bool Collidable { get; set; }
    public bool IsSolid { get; }
    public bool Active { get; set; }

    void Update(GameTime gameTime);
    void Draw(SpriteBatch spriteBatch);
    bool Collides(IEntity other);
}
```

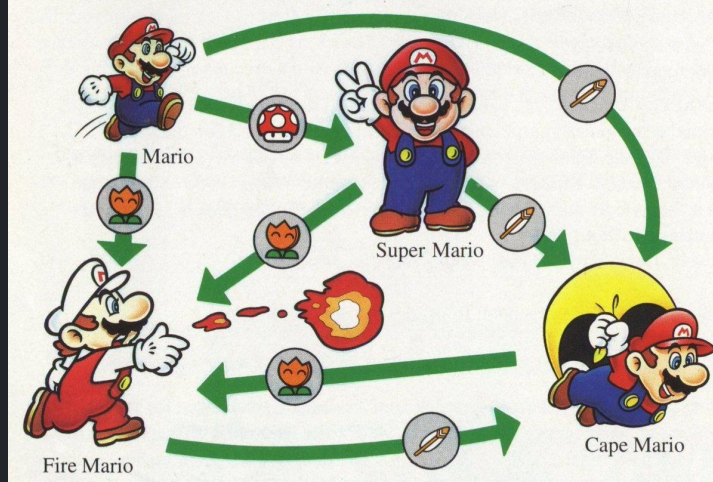
- We iterate over all entities in the world at the same time
 - Made possible by the update method pattern!
- Collisions can be done entity-to-entity
 - Each entity decides how to handle collisions
 - Since entities often move, we cannot do an instant lookup check like for tilemap collisions

Add a Custom Level Maker

- Varying ground level and pillar/pit width
- Platforms
- Multiple types of enemies
- A goal flag
 - When the player touches the flag at the end of the level, they proceed to the next level
 - As the levels are procedural, they can progressively get longer and harder
 - Create random keys and locks (same color) that spawn in each level; when the player gets the key, they can unlock the lock, which should spawn the goal flag.

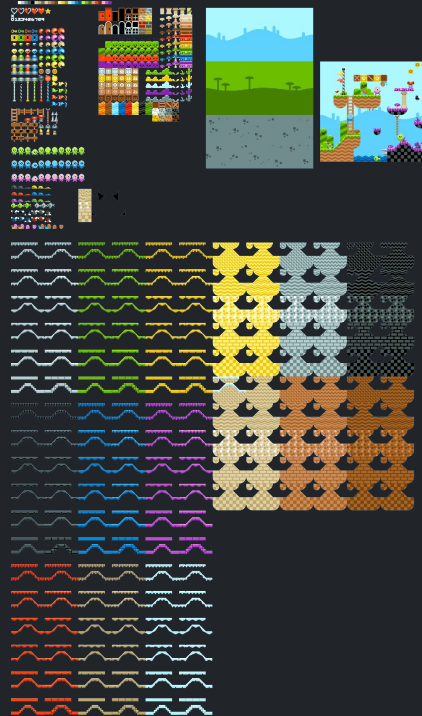
Add Powerups to the Game

- Effectively an Entity that changes some state or trait of the player
 - A star may flip an “invincible” flag on the player and begin an “invincibleDuration” timer
 - A mushroom may trigger a “huge” flag on the player that alters their x, y, width, and height and then scales their sprite



Extra Assets

[All assets](#) (sounds/images) available for the platformer



Project Kick-Off

- Groups should be formed by next week
- Think of a game you want to create
 - Be mindful of scope (creep)
 - Take inspiration from classic games!
 - We will have a session next week dedicated to getting you started!



Midterm Evaluation

